We've defined a fun little class, `QuoteBot`, that provides inspirational quotes on demand. It has a public array property `quoteStash` and a public method `getQuote`. The use of an access level keyword is optional, and default access for all class methods and properties is `public` if it is omitted (but most PHP coding standards will demand that access level is explicitly set).

The implementation of `getQuote()` accesses the `quoteStash` array via the object operator like this: `$this->quoteStash[]`. `$this` is a special variable that can only be used inside the lexical scope (inside the function boundaries) of an object's methods, and it references the object instance executing the method.

Remember at the end of the previous section we had a problem when attempting to use `$this` in a function? The error message was `'Using $this when not in object context'`, and it happened because, despite being used in the lexical scope of *a* function, it was not a function that was defined as a class method. You can only use `$this` during execution of code that exists inside a class method and only when it is being called on an object instance of that class. First you define the method, then you instantiate the class with the `new` keyword. Only then does `$this` have an object context. (We'll look at this idea of *context* in the next section.)

The `QuoteBot` class on its own does nothing. In order to actually use any of the code in `QuoteBot` it is necessary to instantiate it first, which is what happens on the first line `$quoteBot = new QuoteBot()`. Notice that we can call it before its declaration: this is similar to the function hoisting we saw in Chapter 3, "Functions": PHP parses the entire code before running the first line, so it already knows all about `QuoteBot` and therefore is able to build it at the very beginning of the program. There is also a mechanism for finding undeclared classes (autoloading), which we will look at later in this chapter.

I'm hoping that the method declaration for `getQuote()` looks familiar from Chapter 3, "Functions." All of the rules concerning type declaration, scope, parameter handling, and so on that we encountered in that chapter apply to class methods too.

To demonstrate the relationship between class methods, properties, and state, consider the following:

```
$quoteBot1 = new QuoteBot();
$quoteBot2 = new QuoteBot();

$quoteBot2->quoteStash = [
    "Don't get uptight with me, man." . PHP_EOL,
    "I could take double anything you could." . PHP_EOL,
    "Very, very foolish words, man." . PHP_EOL,
];
```